Exp. No: 8

MODEL EVALUATION AND IMPROVEMENT: HYPERPARAMETER TUNING WITH GRID SEARCH AND CROSS-VALIDATION

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import (
  train_test_split,
  KFold,
  cross_val_score,
  GridSearchCV
)
from sklearn.svm import SVC
from sklearn.metrics import (
  accuracy_score,
  classification_report,
  confusion_matrix
)
from sklearn.preprocessing import StandardScaler

# 1. Load the Iris dataset
print("--- Part 1: Hyperparameter Tuning with Grid Search ---")

iris = load_iris()

X = iris.data
y = iris.target
```

```
feature_names = iris.feature_names
target_names = iris.target_names

print("\nDataset Features (X) shape:", X.shape)
print("Dataset Labels (y) shape:", y.shape)
print("Feature Names:", feature_names)
print("Target Names:", target_names)



# 2. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.3,
  random_state=42,
  stratify=y
)

print("\nTraining set size:", X_train.shape[0], "samples")
print("Test set size:", X_test.shape[0], "samples")



# 3. Standardize the features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeatures standardized.")

# 4. Define the model and hyperparameter grid
param_grid = {
  'C': [0.1, 1, 10, 100],
  'gamma': [1, 0.1, 0.01, 0.001],
  'kernel': ['rbf', 'linear']
}
```

```
print("\nHyperparameter grid defined:")

for param, values in param_grid.items():
  print(f" {param}: {values}")



# 5. Perform Grid Search with 5-fold cross-validation
grid_search = GridSearchCV(
  estimator=SVC(),
  param_grid=param_grid,
  cv=5,
  scoring='accuracy',
  verbose=1,
  n_jobs=-1
)

print("\nStarting Grid Search with 5-fold Cross-Validation...")

grid_search.fit(X_train_scaled, y_train)

print("\nGrid Search completed.")



# 6. Display the best hyperparameters and score
print("\nBest hyperparameters found:", grid_search.best_params_)
print(
  "Best cross-validation accuracy:",
  f"{grid_search.best_score_:.4f}"
)



# 7. Evaluate the best model on the test set
best_model = grid_search.best_estimator_

y_pred_tuned = best_model.predict(X_test_scaled)
```

```
test_accuracy_tuned = accuracy_score(y_test, y_pred_tuned)

print(
  "\nTest set accuracy with tuned model:",
  f"{test_accuracy_tuned:.4f}"
)



# 8. Display the classification report
print("\n--- Classification Report for Tuned SVM ---")

print(
  classification_report(
    y_test,
    y_pred_tuned,
    target_names=target_names
  )
)



# 9. Display the confusion matrix
print("\n--- Confusion Matrix for Tuned SVM ---")

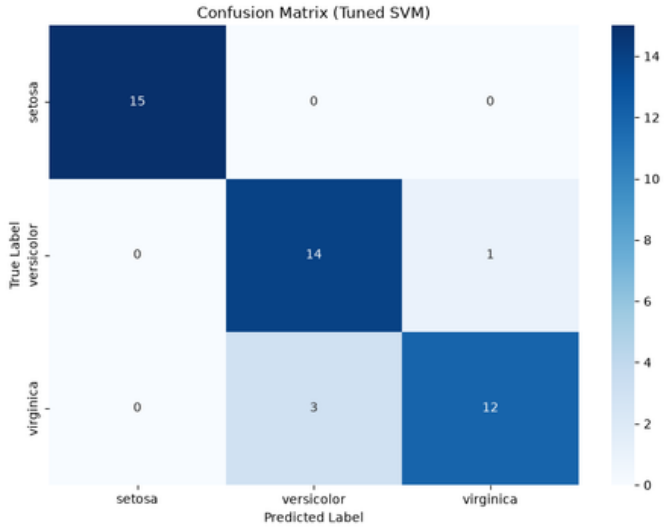
cm_tuned = confusion_matrix(y_test, y_pred_tuned)

plt.figure(figsize=(8, 6))

sns.heatmap(
  cm_tuned,
  annot=True,
  fmt='d',
  cmap='Blues',
  xticklabels=target_names,
  yticklabels=target_names
)
```

```
plt.title('Confusion Matrix (Tuned SVM)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()
```

Output:

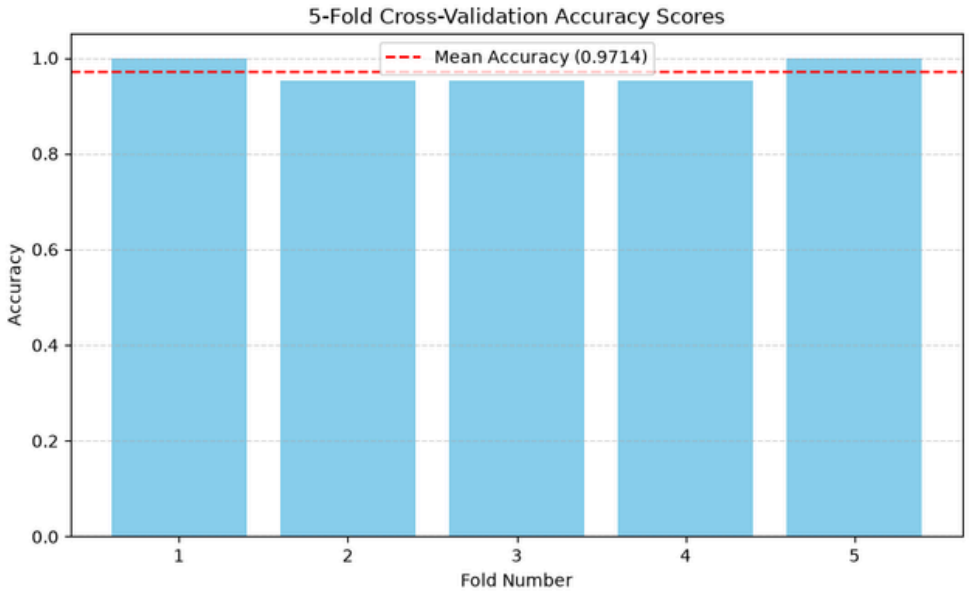



```
--- Top 5 Grid Search Results ---
 param_C  param_gamma param_kernel  mean_test_score  rank_test_score
   1.0       0.100        rbf       0.980952                1
 100.0       0.100      linear       0.980952                1
 100.0       0.001      linear       0.980952                1
 100.0       0.010      linear       0.980952                1
 100.0       1.000      linear       0.980952                1

--- Part 2: Cross-Validation Techniques (k-fold) ---

Performing 5-fold cross-validation...

Cross-validation scores for each fold: [1.         0.95238095
0.95238095 0.95238095 1.        ]
Mean cross-validation accuracy: 0.9714
Standard deviation of cross-validation accuracy: 0.0233
```



--- Why is Cross-Validation Important? ---
1. More Reliable Performance Estimate: Reduces dependence on a single split.
2. Better Generalization: Helps assess performance on unseen data.
3. Efficient Data Usage: Each sample can be used for training and validation.
4. Detects Model Instability: Variation in scores shows performance variability.

Experiment completed successfully!